

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf_49c1018a-e67\agent-ab503b7f977bc4b5c.jsonl`
- SHA-256: `4a9a914042718db082f5b4b18eecd5e4971c16d830a56fd81b7dedd9c01afc66`
- Source modified: `2026-06-18T01:42:59+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`
```

Transcript

1. user (2026-06-18T01:40:32.137Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.
```

CANDIDATE:

```
{
  "title": "Migration ladder not crash-safe: a mid-block interruption in the v3/v4 ALTER steps permanently bricks the DB",
  "file": "backend/infrastructure/database.py",
  "line": "156-157 (v3) and 168-169 (v4); failure window is between each ALTER and its own `PRAGMA user_version = N` at lines 159 and 178",
  "trigger": "Open a pre-v3 (or pre-v4) DB so the v3/v4 migration block runs, and have the process interrupted AFTER one of the non-idempotent `ALTER TABLE ... ADD COLUMN` statements commits but BEFORE that block's `PRAGMA user_version = N` is reached. Realistic causes: SIGKILL/power loss during the one-time upgrade, `disk I/O error`, or an `OperationalError` (e.g. busy_timeout exhausted) raised by a later statement in the same block. Concretely reproduced: start at user_version=3 with `videos` lacking `user_id`; run the v4 block; inject a failure right after `ALTER TABLE videos ADD COLUMN user_id`. Because Python's sqlite3 auto-commits DDL (across the supported 3.10-3.13 range), the column persists but user_version stays 3. On the NEXT open, `_init_db` sees version<4, re-runs `ALTER TABLE videos ADD COLUMN user_id`, and raises `sqlite3.OperationalError: duplicate column name: user_id`. `_init_db` is called from `__init__`, so every subsequent Database(db_path) construction throws and the app can never start against that DB.",
  "why_wrong": "A `PRAGMA user_version` migration ladder is expected to be crash-safe / re-runnable: a block must be re-applicable when interrupted before its version bump. The v1, v2, and v5 blocks honor this (all DDL is `CREATE ... IF NOT EXISTS`), but the v3 and v4 blocks use bare `ALTER TABLE ... ADD COLUMN` (no IF-NOT-EXISTS equivalent, no existence guard). The `with conn:` scope in `_connect` does NOT protect them, because in CPython's sqlite3 each ALTER/CREATE DDL statement commits independently of the surrounding block, so a half-applied block leaves the column present while user_version is unbumped. There is no rollback to a clean v(N-1) state and no recovery path; the DB is permanently unopenable.",
  "severity": "medium",
  "proposed_fix": "Make the v3/v4 ALTERs idempotent so a re-run after an interrupted upgrade is safe. Simplest: before each ALTER, check the column is absent, e.g. helper `def _has_col(cur, table, col): return col in {r[1] for r in cur.execute(f'PRAGMA table_info({table})')}` and guard `if not _has_col(cursor, 'jobs', 'policy'): cursor.execute('ALTER TABLE jobs ADD COLUMN policy TEXT')` (same for next_poll_at, videos.user_id, candidate_segments.user_id). Alternatively, scope each migration block in its own explicit BEGIN/COMMIT and only bump user_version inside
```

the same committed unit, or catch the 'duplicate column name' OperationalError per-ALTER and continue. The IF-NOT-EXISTS index/table statements already in those blocks are fine.",
"confidence": 0.7,
"klass": "SQLite migration ladder & transactions"
}

Read C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/backend/infrastructure/database.py and related code/tests READ-ONLY before judging.

LENS ? REACHABILITY: prove a real path reaches it, or set real=false.

2. assistant (2026-06-18T01:40:34.863Z)

I'll analyze this candidate bug about the migration ladder's crash-safety. Let me read the actual code first.

3. user (2026-06-18T01:40:35.701Z)

```
1     import sqlite3
2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
47             with self._connect() as conn:
```

```

47         cursor = conn.cursor()
48
49     # Videos Table
50     cursor.execute("""
51         CREATE TABLE IF NOT EXISTS videos (
52             id TEXT PRIMARY KEY,
53             filepath TEXT NOT NULL,
54             processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
55             status TEXT NOT NULL,
56             validation_results TEXT
57         )
58     """)
59
60     # Play Segments Table (Extensible metadata JSON)
61     cursor.execute("""
62         CREATE TABLE IF NOT EXISTS play_segments (
63             id INTEGER PRIMARY KEY AUTOINCREMENT,
64             video_id TEXT NOT NULL,
65             start_time REAL NOT NULL,
66             end_time REAL NOT NULL,
67             duration REAL NOT NULL,
68             server TEXT,
69             receiver TEXT,
70             metadata TEXT,
71             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72         )
73     """)
74
75     # Events Table
76     cursor.execute("""
77         CREATE TABLE IF NOT EXISTS events (
78             id INTEGER PRIMARY KEY AUTOINCREMENT,
79             segment_id INTEGER NOT NULL,
80             timestamp REAL NOT NULL,
81             type TEXT NOT NULL,
82             player TEXT,
83             details TEXT,
84             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
85 CASCADE
86         )
87     """)
88
89     # Versioned migrations live here. PRAGMA user_version is the schema
90     # contract ? bump it for every change and add an ordered `if version < N`
91     # block below. Tests assert the expected version, so accidental drift fails
92     CI.
93     version = cursor.execute("PRAGMA user_version").fetchone()[0]
94
95     if version < 1:
96         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
97         # Common fields are columns; detector-specific metrics go in `stats`
98         JSON,
99         # so a new segmenter reuses this table by setting its own
100         `detector`/`stats`.
101         cursor.execute("""
102             CREATE TABLE IF NOT EXISTS candidate_segments (
103                 id INTEGER PRIMARY KEY AUTOINCREMENT,
104                 video_id TEXT NOT NULL,
105                 detector TEXT NOT NULL,
106                 chunk_index INTEGER,
107                 start_time REAL NOT NULL,
108                 end_time REAL NOT NULL,
109                 duration REAL NOT NULL,
110                 confidence REAL,
111                 stats TEXT,

```

```

108         detection_params TEXT,
109         confirmed_segment_id INTEGER,
110         human_label TEXT,
111         notes TEXT,
112         created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114         FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115     )
116 """)
117 cursor.execute("""
118     CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119     ON candidate_segments(video_id, detector)
120 """)
121 cursor.execute("PRAGMA user_version = 1")
122
123 if version < 2:
124     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted
125     # /api/process request. `status` is the EXECUTION state of the job
126     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127     # that failed validation) lives inside `result` JSON as
result["status"].
128     # `result` is the full sync-era response dict (incl. report paths), so
the
129     # observability report is the job's artifact, per the plan.
130     cursor.execute("""
131         CREATE TABLE IF NOT EXISTS jobs (
132             id TEXT PRIMARY KEY,
133             video_path TEXT NOT NULL,
134             video_id TEXT,
135             segmenter TEXT,
136             player_pool TEXT,
137             max_frames INTEGER,
138             status TEXT NOT NULL DEFAULT 'queued',
139             progress TEXT,
140             error TEXT,
141             result TEXT,
142             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143             started_at TIMESTAMP,
144             finished_at TIMESTAMP
145         )
146     """)
147     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148     cursor.execute("PRAGMA user_version = 2")
149
150 if version < 3:
151     # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152     # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153     # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154     # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155     # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156     cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157     cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158     cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159     cursor.execute("PRAGMA user_version = 3")
160
161 if version < 4:
162     # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a

```

```

NULLABLE
163 # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164 # path can scope rows to an owner. **NULL = local install = byte-
identical to
165 # today** ? every existing row stays NULL and every existing query
166 # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167 # Additive ONLY: no served-behavior change rides on this slice.
168 cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169 cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170 # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171 # of index churn while making the eventual per-user lookups cheap.
172 cursor.execute(
173     "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174     "WHERE user_id IS NOT NULL")
175 cursor.execute(
176     "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177     "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178 cursor.execute("PRAGMA user_version = 4")
179
180 if version < 5:
181     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
182     # version): the resolved per-user `fusion_config` overlay + an INLINE
183     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
evidence
184     # and provenance that earned it. `is_active` pins the one row the
serving bridge
185     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
This slice
186     # only persists/loads it ? NOTHING reads it on the served path yet.
187     #
188     #   user_id           ? owner of the model (NULL allowed = the local
install)
189     #   version           ? monotonic per-user model version
190     #   baseline_version  ? the shared baseline this was fit against
(upgrade switch)
191     #   feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
MAJOR re-fit gate
192     #   fusion_config      ? JSON per-user FusionConfig overlay (cue_flags
/ thresholds)
193     #   classifier_json    ? JSON LogisticRegression.to_dict() (None =
config-only model)
194     #   promotion_evidence ? JSON per_user_gate PromotionDecision (why it
was promoted)
195     #   n_videos_at_fit    ? held-out-user corpus size when fit (min_n
trust floor input)
196     #   is_active         ? the one resolved row for this user (partial-
unique below)
197     cursor.execute("""
198         CREATE TABLE IF NOT EXISTS user_models (
199             id INTEGER PRIMARY KEY AUTOINCREMENT,
200             user_id TEXT,
201             version INTEGER NOT NULL DEFAULT 1,
202             baseline_version TEXT,
203             feature_schema_hash TEXT,
204             fusion_config TEXT,
205             classifier_json TEXT,
206             promotion_evidence TEXT,
207             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
208             is_active INTEGER NOT NULL DEFAULT 0,
209             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

```

```

210         )
211     """)
212     # One model version per user (NULLs compare distinct in SQLite, so the
local
213     # install can still carry multiple versioned rows; the active-row guard
below
214     # is what enforces a single served model).
215     cursor.execute("""
216         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
217         ON user_models(user_id, version)
218     """)
219     # At most ONE active model per user: a partial-unique index over the
active rows.
220     # (SQLite treats NULL user_id rows as distinct, so the local install's
single

```

4. assistant (2026-06-18T01:40:38.780Z)

Let me read the rest of the migration block and understand the full picture.

5. user (2026-06-18T01:40:39.975Z)

```

220     # (SQLite treats NULL user_id rows as distinct, so the local install's
single
221     # active row is still guarded ? there is one local user.)
222     cursor.execute("""
223         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
224         ON user_models(user_id) WHERE is_active = 1
225     """)
226     cursor.execute("PRAGMA user_version = 5")
227     # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
228
229     conn.commit()
230
231     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
232         """Register or update a video row WITHOUT touching its child segments.
233
234         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
235         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
236         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
237         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
238         row in place; clearing segments is now an explicit, deliberate operation
239         (clear_video_data / prune_segments).
240         """
241         try:
242             with self._connect() as conn:
243                 conn.cursor().execute(
244                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
245                     "ON CONFLICT(id) DO UPDATE SET "
246                     "filepath=excluded.filepath, status=excluded.status, "
247                     "validation_results=excluded.validation_results",
248                     (video_id, filepath, status, json.dumps(validation_results))
249                 )
250                 conn.commit()
251             return True
252         except Exception as e:
253             logger.error(f"Failed to add video: {e}")
254             return False
255

```

```

256     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
257         """Return (max play_segment id, max candidate_segment id) for a video (0 if
258         none).
259         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
260         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
261         """
262         with self._connect() as conn:
263             cur = conn.cursor()
264             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
265             video_id = ?",
266                             (video_id,)).fetchone()[0]
267             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
268             video_id = ?",
269                             (video_id,)).fetchone()[0]
270             return int(p), int(c)
271
272     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
273                       play_after: Optional[int] = None, cand_upto: Optional[int] =
274                       None,
275                       cand_after: Optional[int] = None) -> None:
276         """Delete play/candidate segments by id range (events cascade from
277         play_segments).
278         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
279         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
280         rows
281         and keep the OLD ones when the run failed/produced nothing (`*_after`).
282         """
283         with self._connect() as conn:

```

6. assistant (2026-06-18T01:40:44.070Z)

Now I have a complete picture of the migration ladder. Let me analyze the transactional semantics carefully. The key claim in the candidate is that DDL auto-commits independently in CPython's sqlite3, so a half-applied v3/v4 block leaves the column present while `user\_version` stays unbumped.

Let me verify the actual transaction behavior. This is the crux. The `\_connect` wraps everything in `with conn:`. Let me check the Python version and sqlite3 transaction semantics carefully.

7. user (2026-06-18T01:40:46.113Z)

```

Python 3.13.13
<string>:1: DeprecationWarning: version is deprecated and will be removed in Python 3.14
sqlite3 2.6.0 sqlite 3.50.4

```

8. assistant (2026-06-18T01:40:49.477Z)

Python 3.13. This is significant. The candidate explicitly claims behavior "across the supported 3.10-3.13 range." But Python 3.12 changed `sqlite3` transaction handling significantly. Let me test the actual behavior empirically since that's the load-bearing claim.

9. user (2026-06-18T01:40:59.931Z)

```

default isolation_level:
caught: simulated crash AFTER first ALTER, BEFORE PRAGMA user_version=4
after with-block-abort: videos columns = ['id', 'filepath', 'status', 'validation_results',
'user_id']
after with-block-abort: user_version = 3

```

10. assistant (2026-06-18T01:41:04.599Z)

This is interesting and partially contradicts the candidate's reasoning. The default

`isolation\_level` is empty string `` (the legacy implicit-transaction mode), and the ALTER persisted while the version stayed at 3 ? but NOT because "DDL auto-commits independently." Let me dig into WHY it persisted, because the candidate's mechanism claim matters for whether this is real.

The `with conn:` block raised, which should `rollback()`. Yet the column persisted. Let me investigate the mechanism more precisely ? this is the key to whether the bug is real or whether a `BEGIN`/rollback would actually save it.

11. user (2026-06-18T01:41:10.737Z)

```
isolation_level: ''
in_transaction before ALTER: False
in_transaction after ALTER: False
in_transaction after rollback: False
columns after explicit rollback: ['id', 'user_id']
```

12. assistant (2026-06-18T01:41:15.867Z)

This nails the mechanism. With the default `isolation\_level=''` in Python 3.13, after the `ALTER`, `conn.in\_transaction` is `False` ? meaning the sqlite3 module did NOT open a transaction for the DDL, so the subsequent `rollback()` is a no-op and the column persists. This confirms the candidate's core claim about the mechanism (DDL is auto-committed / not wrapped in the transaction), even on Python 3.13.

So the `with conn:` scope in `\_connect` does NOT protect the v3/v4 ALTERs. Now let me verify the second half: that on the NEXT open, `\_init\_db` re-runs the ALTER and raises `duplicate column name`, bricking the DB permanently. Let me simulate the full real path through `\_init\_db`.

13. user (2026-06-18T01:41:24.998Z)

```
partial state ready: videos has user_id, version still 3
Database() RAISED: OperationalError - duplicate column name: user_id
```

14. assistant (2026-06-18T01:41:34.240Z)

Confirmed end-to-end against the real `Database` class. The partial-v4 DB (videos has `user\_id`, version still 3) causes `Database(db\_path)` to raise `sqlite3.OperationalError: duplicate column name: user\_id` on every subsequent construction ? the DB is permanently unopenable.

Now let me assess the REACHABILITY lens rigorously, which is what I'm specifically tasked to judge. The mechanism is real, but is there a *real path* that reaches the failure window? The candidate lists triggers: SIGKILL/power loss, disk I/O error, or an `OperationalError` from a later statement in the same block. Let me check two things:

1. Whether the v3/v4 blocks could ever actually run in practice (i.e., does any real deployed DB sit at version < 4 such that the block runs at all).
2. Whether there's a realistic in-process failure between the ALTER and the version bump (not just SIGKILL, which is somewhat out of scope for "code defect").

Let me first check whether the failure window between the two ALTERs in the same block is reachable via a *code* path. Look at the v3 block: two ALTERs then a CREATE INDEX then the version bump. If the second ALTER or the CREATE INDEX fails after the first ALTER committed, the block aborts with version still at the prior value.

15. user (2026-06-18T01:41:41.448Z)

```
partial v3 ready: jobs.policy present, next_poll_at absent, version=2
RAISED: OperationalError - duplicate column name: policy
```

16. assistant (2026-06-18T01:41:48.247Z)

Same brick for the v3 block. Now let me weigh the REACHABILITY honestly. The mechanism and bricking are proven. The question is whether a *realistic, code-level* trigger reaches the

failure window ? not just an external SIGKILL/power loss (which would brick almost any non-atomic migration system and is arguably an inherent risk rather than a code defect).

Let me check whether the v3/v4 blocks even run for any real deployment ? i.e., is there an existing on-disk DB that sits below version 4? This matters: if the async-job model and personalization are brand-new and no v2/v3 DBs exist in the wild yet, the migration block would only run during a fresh `CREATE` followed immediately by ALTERs in the same `\_init\_db` call, where all blocks run sequentially in one process.

Let me check the tests and how the migration ladder is exercised, and whether there's a realistic mid-block in-process failure.

17. user (2026-06-18T01:41:53.762Z)

fresh DB version: 5
re-open OK, version: 5

18. assistant (2026-06-18T01:42:01.880Z)

As expected: on a fresh DB, all blocks run once in a single `\_init\_db` call, end at version 5, and re-opening is fine because every block is skipped. So in normal operation there's no problem ? the failure requires an *interruption during a one-time partial upgrade*.

Now let me look for the tests covering this migration ladder to understand whether maintainers treat crash-safety as a contract, and check whether there's any existing guard or documented expectation.

19. user (2026-06-18T01:42:02.529Z)

Found 9 files
.claude\worktrees\hunt-base\tools\generate_highlights.py
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py
.claude\worktrees\hunt-base\tests\test_golden_real_fixtures.py
.claude\worktrees\hunt-base\tests\test_golden_fixtures.py
.claude\worktrees\hunt-base\tests\test_generate_highlights.py
.claude\worktrees\hunt-base\tests\test_database.py
.claude\worktrees\hunt-base\tests\test_async_jobs.py
.claude\worktrees\hunt-base\backend\infrastructure\database.py
.claude\worktrees\hunt-base\backend\config\models.py

20. user (2026-06-18T01:42:02.787Z)

Found 3 files
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py
.claude\worktrees\hunt-base\tests\test_database.py
.claude\worktrees\hunt-base\tests\test_async_jobs.py

21. user (2026-06-18T01:42:06.268Z)

```
60:     "user_id", # v4: personalization groundwork (nullable; NULL = local install)
84:     version = conn.execute("PRAGMA user_version").fetchone()[0]
86:     # v5 = per user model store (PERSONALIZATION_PLAN.md §2). Bump with each migration.
90: def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
100:     # ...and running init again must be a no-op (idempotent).
106:     version = conn.execute("PRAGMA user_version").fetchone()[0]
169: # --- v4: personalization groundwork (nullable user_id; NULL = local = unchanged) ----
178: def test_v4_user_id_columns_present_and_nullable(tmp_path):
181:     assert "user_id" in _table_cols(db_path, "videos")
182:     assert "user_id" in _table_cols(db_path, "candidate_segments")
188:     assert by_name["user_id"][3] == 0, f"{table}.user_id must be nullable"
192: def test_v4_existing_writers_leave_user_id_null(tmp_path):
193:     """NULL user_id back-compat: add_video / add_candidate_segment never set user_id, so
every
194:     row a current writer produces is local (NULL) ? byte-identical to pre-v4 behaviour."""
```

```

201:     vu = conn.execute("SELECT user_id FROM videos WHERE id = 'vid1']").fetchone()[0]
202:     cu = conn.execute("SELECT user_id FROM candidate_segments WHERE video_id =
'vid1']").fetchone()[0]
206:     # Existing readers are unaffected (no user_id filter in the query path).
210: def test_migration_v3_db_upgrades_to_v5_preserving_rows(tmp_path):
211:     """A real v3 DB (jobs + candidates already populated) upgrades cleanly to v5: the new
212:     columns/tables appear, user_id back-fills to NULL, and prior rows survive the ALTERs."""
213:     db_path = tmp_path / "legacy_v3.db"
214:     db = Database(str(db_path)) # fresh DB is born at the current version; simulate a v3
state
215:     # Seed pre-upgrade data, then force the version back to 3 and re-open to run v4+v5.
222:     # Recreate v3-shaped tables WITHOUT user_id by rebuilding (simplest: just rewind version
223:     # and drop the v4/v5 artifacts that a true v3 DB wouldn't have).
224:     conn.execute("ALTER TABLE videos DROP COLUMN user_id")
225:     conn.execute("ALTER TABLE candidate_segments DROP COLUMN user_id")
226:     conn.execute("PRAGMA user_version = 3")
229:     assert "user_id" not in _table_cols(db_path, "videos")
231:     # Re-open: the migration ladder must carry the v3 DB up to v5 without data loss.
234:     assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
238:     assert "user_id" in _table_cols(db_path, "videos")
239:     assert "user_id" in _table_cols(db_path, "candidate_segments")
254:     "id", "user_id", "version", "baseline_version", "feature_schema_hash",
274:     assert m["user_id"] == "user-1"
294: def test_upsert_is_idempotent_on_user_version(tmp_path):
301:     # Same (user, version) ? same row updated in place, not a duplicate.
309:     "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-3'").fetchone()[0]
325:     "SELECT COUNT(*) FROM user_models WHERE user_id = 'user-4' AND is_active =
1").fetchone()[0]
330: def test_null_user_id_local_install_round_trip(tmp_path):
331:     """NULL user_id = the local install. upsert/get must treat IS NULL correctly (not
'=NULL')."""
337:     assert m["user_id"] is None

```

22. assistant (2026-06-18T01:42:11.462Z)

The tests confirm the maintainer's intent and contract.

`test\_migration\_v3\_db\_upgrades\_to\_v5\_preserving\_rows` (line 210) explicitly tests the *happy-path* upgrade of a v3 DB to v5 ? proving the maintainer cares that real legacy DBs upgrade through these exact ALTERs. But crucially, there is **no** test for an interrupted/partial migration* ? the crash-safety contract the candidate alleges is **not** covered by any test (consistent with the candidate's "no recovery path" claim).

Let me read that test block fully and also check whether there's any documented expectation that the ladder must be re-runnable / crash-safe, to confirm a maintainer would call this a defect.

23. user (2026-06-18T01:42:12.926Z)

```

90     def test_init_db_idempotent_and_upgrades_old_db(tmp_path):
91         db_path = tmp_path / "legacy.db"
92         # Simulate a pre-v1 database that only has the old tables and no version.
93         conn = sqlite3.connect(str(db_path))
94         conn.execute("CREATE TABLE videos (id TEXT PRIMARY KEY, filepath TEXT, status
TEXT)")
95         conn.commit()
96         conn.close()
97
98         # Opening with the new Database should add candidate_segments without error...
99         Database(str(db_path))
100        # ...and running init again must be a no-op (idempotent).
101        Database(str(db_path))
102
103        conn = sqlite3.connect(str(db_path))
104        tables = [r[0] for r in conn.execute(
105            "SELECT name FROM sqlite_master WHERE type='table'").fetchall()]
106        version = conn.execute("PRAGMA user_version").fetchone()[0]

```

```

107         conn.close()
108         assert "candidate_segments" in tables
109         assert "jobs" in tables # v2 (async job model, #16)
110         assert "user_models" in tables # v5 (per-user model store)
111         assert version == 5 # v5 = per-user model store (PERSONALIZATION_PLAN.md §2)
112
113
114     def test_candidate_round_trip(tmp_path):
115         db = Database(str(tmp_path / "test.db"))
116         db.add_video("vid1", "vid1.mp4", "processed", [])
117
118         cid = db.add_candidate_segment(
119             "vid1", "yolo_hybrid", 10.0, 25.0, chunk_index=0, confidence=0.42,

```

24. assistant (2026-06-18T01:42:31.234Z)

I now have everything I need to judge this rigorously.

Let me synthesize the assessment:

****Mechanism ? CONFIRMED.**** Empirically verified on the repo's actual Python 3.13 / SQLite 3.50: with the default `isolation_level=''`, `conn.in_transaction` is `False` immediately after an `ALTER TABLE ... ADD COLUMN` ? sqlite3 does NOT wrap DDL in the implicit transaction, so the surrounding `with conn:` rollback is a no-op and the column persists even when the block aborts before its `PRAGMA user_version = N`. The candidate's mechanism claim holds (and is even more robustly true than its hedge about the 3.10?3.13 range).

****Bricking ? CONFIRMED end-to-end**** against the real `Database` class: a partial-v3 (jobs.policy present, version=2) or partial-v4 (videos.user_id present, version=3) DB makes `Database(db_path)` raise `sqlite3.OperationalError: duplicate column name: ...` on every subsequent construction. Since `_init_db` runs in `__init__`, the app can never open that DB again. No recovery path exists.

****Maintainer-defect ? YES.**** `test_migration_v3_db_upgrades_to_v5_preserving_rows` proves the maintainer treats upgrading real legacy v2/v3 DBs through these exact ALTERs as a supported contract, while v1/v2/v5 blocks are deliberately written `IF NOT EXISTS` (re-runnable) and only v3/v4 use bare non-idempotent ALTERs. The asymmetry plus the explicit legacy-upgrade test means a maintainer would recognize the non-idempotent ALTER as a real crash-safety defect.

****REACHABILITY (my assigned lens) ? the honest weak point.**** The failure window is reachable only during a **one-time partial upgrade** of an **already-existing** v2/v3 on-disk DB. It requires:

- A real legacy DB at version < 4 (the async-job model and personalization are recent; per repo state "no platform implementation has started," so such DBs are rare-to-nonexistent in the wild today, but the migration path is shipped and tested as supported).
- An interruption strictly between the committed ALTER and the version bump. In-process code triggers within that window are thin: the only statements between the first ALTER and the bump are another ALTER / CREATE INDEX (which only fail if already-partial) or a busy-timeout `OperationalError`. The realistic trigger is genuinely external (SIGKILL / power loss / disk-full / I/O error) ? which is somewhat inherent to any non-atomic migration, not unique to this code.

So: the bug is ****real**** (the code is genuinely non-crash-safe where sibling blocks are, and a maintainer would fix it), and a path ****does**** reach it, but that path needs both a legacy sub-v4 DB and an interrupt in a narrow window ? predominantly an external crash rather than a pure in-process logic trigger. It is not already guarded or tested (no partial-migration test exists). Severity medium is fair (one-time, permanent brick, no data-loss-of-content but total lockout; low frequency).

I'll mark `reachable=true` because a concrete, demonstrated path exists (I reproduced the brick end-to-end), while being explicit in the reasoning that the trigger within the window is chiefly an external interruption and the prerequisite legacy DB is currently uncommon.

25. user (2026-06-18T01:42:47.286Z)

Structured output provided successfully

26. assistant (2026-06-18T01:42:59.762Z)

I've completed the adversarial reachability assessment of this candidate.

****Verdict: real defect, reachable, not guarded ? confirmed end-to-end.****

Key findings, all empirically reproduced against the actual `Database` class on this repo's Python 3.13 / SQLite 3.50:

1. ****Mechanism holds.**** With the default `isolation\_level=''`, `conn.in\_transaction` is `False` immediately after `ALTER TABLE ... ADD COLUMN`, so the `with conn:` rollback in `\_connect` (lines 24-25) is a no-op for DDL. A half-applied v3/v4 block leaves the column present while `PRAGMA user\_version` stays unbumped.

2. ****Brick reproduced.**** A partial-v3 DB (`jobs.policy` present, version=2) and a partial-v4 DB (`videos.user\_id` present, version=3) each make `Database(db\_path)` raise `sqlite3.OperationalError: duplicate column name: ...` on every construction. Since `\_init\_db` is in `\_\_init\_\_`, the DB becomes permanently unopenable with no recovery path.

3. ****Maintainer-defect.**** Only v3/v4 (lines 156-157, 168-169) use bare non-idempotent ALTERs; v1/v2/v5 are deliberately `IF NOT EXISTS`. `tests/test\_database.py:210` proves the maintainer treats the v3?v5 upgrade through these ALTERs as supported ? yet no test covers a partial/interrupted migration.

****Honest reachability caveat (my assigned lens):**** the window is hit only during a one-time partial upgrade of an existing sub-v4 on-disk DB, and the realistic in-window trigger is external (SIGKILL/power-loss/disk-full/I-O error) rather than a pure in-process logic fault. Such legacy DBs are currently uncommon (per repo state, the platform/async-job track hasn't been deployed). So: real and reachable, medium severity, low real-world frequency today. Confidence ~0.7, discounted only for those reachability prerequisites.

Relevant files:

- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/backend/infrastructure/database.py` (v3 block 150-159, v4 block 161-178)
- `C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base/tests/test\_database.py` (happy-path upgrade test at 210; no partial-migration coverage)